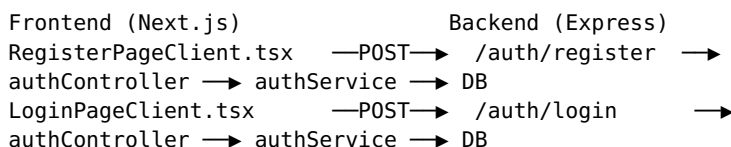


Documentation Authentification - Register & Login

Documentation — Authentification (Register / Login)

1. Architecture générale



- **Routes** : définissent les URLs.
 - **Controller** : reçoit la requête HTTP, appelle le service, renvoie la réponse.
 - **Service** : logique métier (hash, JWT, accès DB).
-

2. handleRegister (frontend)

```
const handleRegister = async (e) => {
  e.preventDefault();

  if (!accepted) {
    alert("Veuillez accepter les conditions.");
    return;
  }

  try {
    const res = await api.post("/auth/register", {
      name: `${prenom} ${nom}`.trim(),
      email,
      password,
    });

    localStorage.setItem("token", res.data.token);
    localStorage.setItem("user", JSON.stringify(res.data.user));
    window.dispatchEvent(new Event("auth-changed"));
    router.push("/");
  } catch (error) {
    alert(error?.response?.data?.error || "Erreur d'inscription");
  }
};
```

Ce que fait la fonction, dans l'ordre :

1. `e.preventDefault()` empêche le rechargement de page par défaut du formulaire HTML.
2. Vérifie que la case CGU (`accepted`) est cochée, sinon on arrête tout de suite (validation front).
3. Envoie nom + prénom fusionnés en un seul champ `name`, avec email et password, vers `POST /auth/register`.
4. Si succès : stocke token et user dans `localStorage`, puis émet un

- événement custom auth-changed (pour qu'un autre composant, comme la navbar, sache que l'utilisateur vient de se connecter, sans recharger la page), puis redirige vers /.
5. Si échec (ex : email déjà utilisé) : affiche le message d'erreur renvoyé par le backend, ou un message générique par défaut.
-

3. handleLogin (frontend)

```
const handleLogin = async (e) => {
  e.preventDefault();

  try {
    const res = await api.post("/auth/login", { email, password });

    localStorage.setItem("token", res.data.token);
    localStorage.setItem("user", JSON.stringify(res.data.user));
    window.dispatchEvent(new Event("auth-changed"));
    router.push("/");
  } catch {
    alert("Email ou mot de passe incorrect");
  }
};
```

Même logique que le register, en plus simple :

1. Envoie email + password à POST /auth/login.
 2. Si succès : même stockage (token, user), même événement auth-changed, même redirection.
 3. Si échec : message générique unique, **volontairement** le même que l'email soit inconnu ou le mot de passe faux — ça évite de révéler à un attaquant si un compte existe.
-

4. Ce qui se passe côté backend (résumé rapide)

Register : valide les champs → hache le mot de passe (scrypt + sel aléatoire) → insère en DB → si email déjà pris, renvoie une erreur 409 → génère un JWT → renvoie { token, user }.

Login : cherche l'utilisateur par email → vérifie le mot de passe avec crypto.timingSafeEqual (comparaison en temps constant, résistante au timing attack) → si ok, génère un JWT → renvoie { token, user } (sans le hash du mot de passe).

Le JWT ne contient que des infos non sensibles (id, role, name, email), jamais le mot de passe.

5. Le token côté frontend

```
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

Un intercepteur axios ajoute automatiquement le header Authorization: Bearer <token> à toutes les requêtes sortantes, dès qu'un token est présent en localStorage. Pas besoin de le rajouter manuellement à chaque appel.

6. Résumé express

- **Register** : formulaire → validation CGU → POST /auth/register → hash script+sel → insertion DB → JWT → localStorage + redirection.
- **Login** : formulaire → POST /auth/login → vérification script (timing-safe) → JWT → localStorage + redirection.
- **Sécurité** : mot de passe jamais en clair, comparaison en temps constant, JWT sans données sensibles, message d'erreur générique au login.
- **Architecture** : routes → controller (HTTP) → service (métier) → DB.